TECHNICAL DEVELOPMENT SPECIFICATION

Dota 2 Reinforcement Learning & Meta-Discovery System

Evaluation, Architecture, Implementation Plan, Interfaces, Training Protocol & Acceptance Criteria

Status: Engineering-ready blueprint for sandbox / offline research only.

Basis: user-provided “Blueprint Sistem RL & Analisis Meta Dota 2 (Versi Revisi)”.

Important: this specification preserves the source scope limitation to sandbox research. The source explicitly excludes public/ranked automated play. [Source: Blueprint, Section 0, lines 7–24]

Document attribute	Value
Primary objective	Buildable PoC → scalable research platform
Primary algorithm	PPO + LSTM, with optional RND and league self-play
Training scope	Curriculum F0–F5; promotion gates are mandatory
Deployment scope	Private/offline sandbox only
Recommended MVP	F0–F2 before any full-map 5v5 attempt
Engineering principle	Environment correctness before model complexity

1. Executive Technical Assessment

Overall verdict: the revised blueprint is directionally strong, but it is not yet implementation-complete. It describes the right major components—curriculum learning, PPO/LSTM, embeddings, exploration, reward shaping, evaluation and patch resilience—but several critical interfaces and failure controls are underspecified.

Area	Assessment	Severity	Engineering decision
Scope / legal boundary	Clear sandbox-only boundary and mandatory API validation.	Low	Keep as hard gate.
Environment interface	Conceptually correct, but actual Bot API data availability is unverified.	Critical	Build F0 API probe before model design.
PPO + LSTM	Reasonable baseline for temporal decision-making.	Medium	Use as baseline; do not over-scale initially.
Hierarchical actions	Correct direction, but needs explicit action grammar + invalid-action masking.	Critical	Implement structured policy heads + masks.
Hero/item embeddings	Useful for representation, but not sufficient to guarantee zero-retrain new-hero generalization.	High	Treat as robustness mechanism, not a guarantee.
Reward shaping	Better than original, but weights and normalization are unspecified; raw net-worth deltas can dominate.	Critical	Normalize, clip, ablate, and preserve terminal objective.
RND + league	Promising but too many moving parts at once.	High	Enable by ablation sequence, not all at once.
Evaluation	Good observability basics, but win-rate alone is insufficient.	Critical	Add Elo, skill metrics, objective metrics, robustness and regression suite.
Compute plan	Useful upper/lower references but still coarse.	High	Measure env step throughput first; capacity from benchmark.
Meta extraction	Value-head analysis is useful but should be event- and counterfactual-aware.	Medium	Add event attribution and matched comparisons.

The source itself already states the right high-level principle: start small, verify the pipeline, then increase scope. [Source: Blueprint, Section 1, lines 28–32]

2. Design Goals and Non-Goals

2.1 Goals

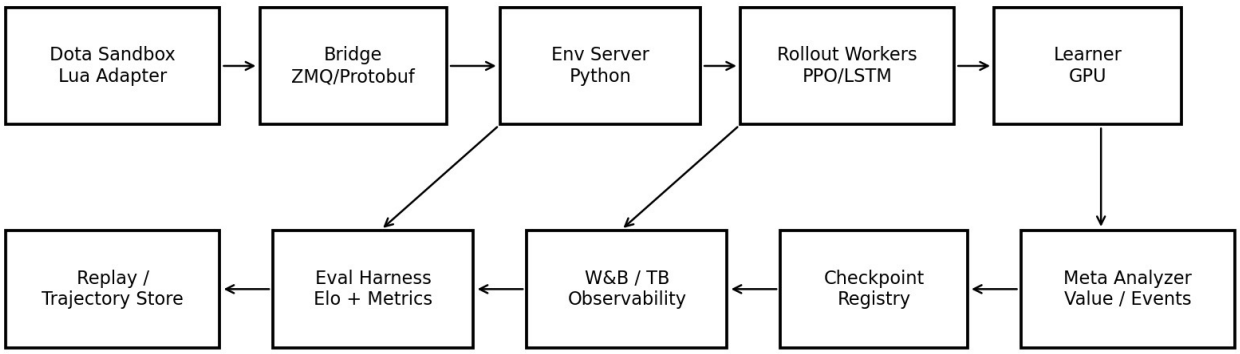
- Create a repeatable RL training environment for Dota 2 sandbox research.
- Train a policy that improves combat, farming, positioning, item/skill timing and objective decisions as curriculum difficulty increases.
- Measure whether the policy discovers strategies that are both higher-performing and robust to opponent changes.
- Produce interpretable “meta candidates” such as build order, timing windows, objective preferences and matchup-specific tactics.
- Support controlled patch migration through schema/version checks and fine-tuning.

2.2 Non-goals

- Automated public/ranked match play.
- Bypassing anti-cheat or obtaining hidden game state unavailable to normal public scripting interfaces.

- Claiming human-level or globally optimal play without controlled benchmark evidence.
- Assuming a fixed 5v5 full-map system is feasible before the environment throughput and data-access gates pass.

3. Target System Architecture



The revised architecture should separate five concerns: environment adapter, environment server, rollout/training, evaluation, and analysis. This makes it possible to replace the Dota runtime with a simulator without rewriting the learner.

3.1 Recommended Runtime Boundaries

Component	Responsibility	Language	MVP status
Lua Adapter	Read state, execute validated actions, emit event markers.	Lua	Required
Bridge	Serialize/transport observations and actions.	Python initially; C++/Rust later	Required
Environment Server	Gym-like reset/step API, validation, timing, reward calculation.	Python	Required
Policy / Learner	PPO, recurrent state, action distributions, value function.	Python/PyTorch	Required
Rollout Manager	Parallel environment workers, seeding, trajectory collection.	Python/Ray or multiprocessing	Required
Evaluator	Fixed benchmark seeds, opponent pools, statistics.	Python	Required
Experiment Store	Configs, checkpoints, metrics, artifacts.	Filesystem + W&B/TensorBoard	Required
Meta Analyzer	Event-aligned value deltas and strategy summaries.	Python	After F2

3.2 Key Design Decision: Delay C++/Rust

The source proposes a C++/Rust ingestion engine. For the PoC, this is not the highest-risk component. Start with Python serialization and profiling. Introduce C++ only after profiling proves parsing/IPC is a bottleneck. This reduces integration surface while environment correctness is still being established.

4. Environment Contract — the Core of the Build

The first deliverable is not the neural network; it is a deterministic-enough environment contract. The source requires a minimal 1-hero prototype to verify which observations are actually accessible through the public Bot API. [Source: Blueprint, Section 0, lines 18–24]

4.1 Observation Schema

```
Observation {
  episode_id: string
  step_id: int64
  sim_time: float32
  patch_id: string
  agent: {
    hp_ratio: float32
    mana_ratio: float32
    level_norm: float32
    gold_norm: float32
    xp_norm: float32
    pos_norm: [float32, float32]
    facing: float32
  }
  abilities: [
    { available: bool, cooldown_ratio: float32, mana_ratio: float32,
      effect_embedding: [float32, ...] }
  ]
  nearby_entities: [
    { entity_id: int32, team: int8, visible: bool,
      pos_norm: [float32,float32], hp_ratio: float32,
      dist_norm: float32, hero_embedding: [float32,...] }
  ]
  inventory: [...]
  objectives: {...}
  masks: {...}
}
```

4.2 Observation Rules

- Normalize quantities by meaningful denominators: HP/max HP, cooldown/max cooldown, distance/relevant range, level/max level.
- Never silently fabricate inaccessible information. Unavailable fields must be represented explicitly as unknown/masked.
- Carry patch_id and schema_version on every episode.
- Use fixed ordering for entities in each observation or an attention/set encoder; do not depend on incidental iteration order.
- Add unit tests for normalization bounds and missing-data behavior.

4.3 Action Contract

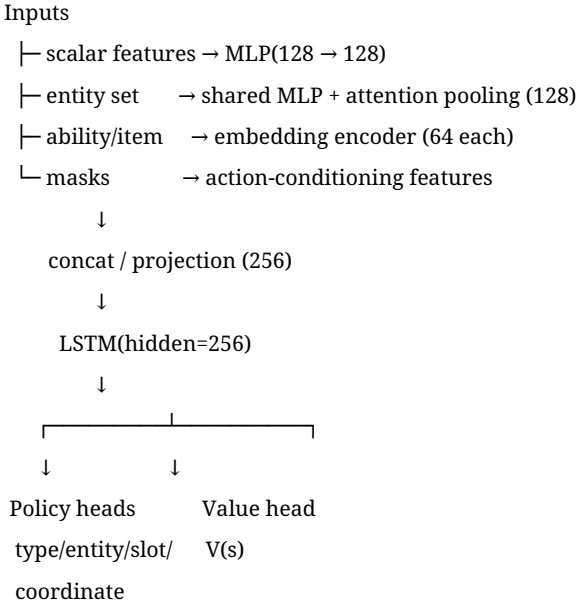
```
Action {
  action_type: MOVE | ATTACK | CAST | ITEM | STOP
  target_entity_id: int32 | -1
  target_xy: [float32, float32] | null
  ability_slot: int8 | -1
  item_slot: int8 | -1
}

ActionMask {
  action_type_mask: [bool, ...]
  entity_mask: [bool, ...]
  ability_mask: [bool, ...]
  item_mask: [bool, ...]
  coordinate_mask: bool
}
```

The action mask is mandatory. Without masking, PPO spends a large fraction of probability mass on impossible actions, making the learning problem materially harder and causing avoidable invalid-action noise.

5. Model Architecture — Buildable PPO/LSTM Baseline

5.1 Recommended MVP Network



Use multi-head categorical distributions for discrete choices and a bounded continuous distribution for target coordinates only when the environment supports meaningful continuous targeting. Avoid a single monolithic categorical over the entire Cartesian action space.

5.2 Recurrent State Handling

- Store hidden/cell state per environment worker.
- Reset recurrent state on episode boundary and after hard environment resets.

- Train with truncated backpropagation through time (TBPTT) over fixed rollout chunks.
- Mask padded sequence timesteps in both policy loss and value loss.
- Log hidden-state norm to detect exploding/vanishing recurrence.

5.3 Hero / Item Embeddings — corrected expectation

The source proposes attribute-based embeddings to improve resilience to new heroes and items. [Source: Blueprint, Section 3B, lines 62–68] The engineering correction is important: embeddings remove the need for fixed one-hot dimensionality, but they do not imply that a newly introduced hero will immediately have optimal behavior without adaptation. The model still needs to learn how attribute combinations map to decisions. Therefore define two targets separately: (1) architectural compatibility and (2) performance generalization.

Test	Pass criterion
Architectural compatibility	New hero/item metadata can be loaded without changing network tensor dimensions.
Zero-shot generalization	New content reaches >= X% of incumbent policy score on controlled scenarios.
Few-shot adaptation	Fine-tuning on a small sample improves to target threshold without catastrophic regression.

6. Reward System — Production-Ready Definition

The source correctly identifies reward hacking risk in the simpler formulation and adds objective control plus terminal win reward. [Source: Blueprint, Section 3D, lines 77–90] For implementation, add normalization and temporal smoothing.

6.1 Reward Components

Signal	Raw source	Normalization	Frequency
Economy	Δ net worth / XP	Per-minute or bounded delta	Per step
Combat	Kill/death events	Small bounded event reward	Event
Structure	Tower HP / objective damage	Normalized by max structure HP	Per step/event
Objectives	Roshan / vision / map objective state	Score delta, bounded	Event / periodic
Positioning	Optional safety/advantage proxy	Potential-based or heavily bounded	Per step
Terminal	Win/Loss	Large fixed terminal reward	Episode end

Do not begin with all terms enabled. Start with the minimum sufficient signal and add terms only after an ablation confirms improvement.

6.2 Recommended Implementation

```
r_t = clip(
    w_eco * z(delta_net_worth)
  + w_xp  * z(delta_xp)
  + w_com * event_combat
  + w_obj * delta_objective
  + w_str * delta_structure
  + w_term * terminal_result,
  r_min, r_max
)
```

```
r_team = (1 - tau) * r_agent + tau * r_team_global
```

Weights must live in versioned configuration, not hard-coded in Python. The source suggests increasing the team blending parameter τ from individual-focused training toward team-focused training. [Source: Blueprint, Section 3D, lines 88–90]

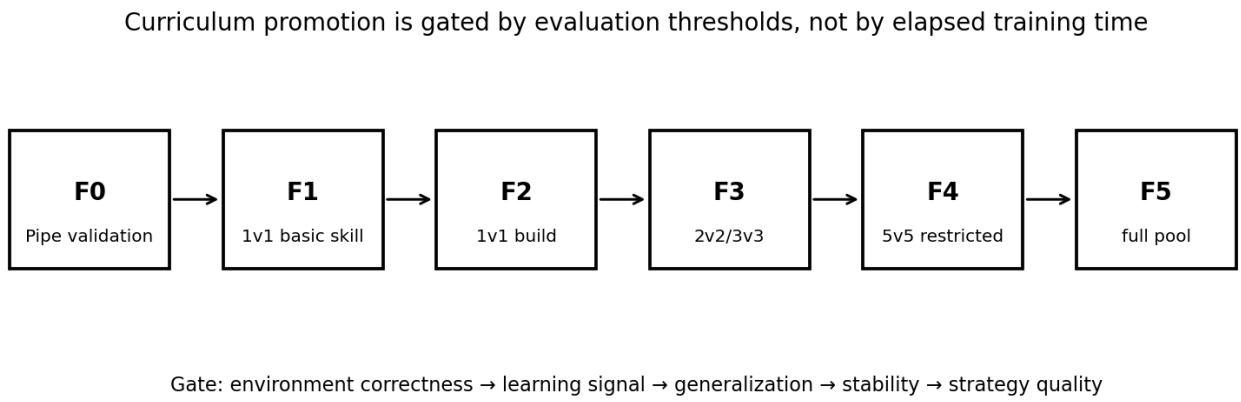
7. Exploration & Self-Play Strategy

The source proposes entropy regularization, league training, RND and curiosity annealing. [Source: Blueprint, Section 3C, lines 70–75] These should be introduced sequentially.

Stage	Exploration mechanism	Why
E0	Entropy only	Establish PPO stability.
E1	Self-play current policy	Test basic opponent adaptation.
E2	Historical policy pool	Reduce cycling and forgetting.
E3	RND	Add novelty pressure only if exploration stalls.
E4	RND annealing	Reduce exploration late in training.

The 80/20 league split in the source should be treated as an initial hypothesis, not a fixed law. The correct ratio is one of the experiment variables and should be selected from evaluation outcomes.

8. Curriculum Learning & Promotion Gates



The source defines six curriculum phases and recommends explicit promotion criteria. [Source: Blueprint, Section 5, lines 106–117] Replace the single “winrate >70%” idea with a multidimensional gate.

Phase	Required metrics	Promotion example
F0	Reset/step correctness, action execution success, reward observability, no desync	>99% valid transitions; zero protocol deadlocks in soak test
F1	Last-hit, deny, survival, combat efficiency, winrate	Stable learning across 3 seeds + target winrate vs fixed bots
F2	Build progression, power-spike timing, winrate	Statistically significant improvement vs baseline + no regression on F1 suite

F3	Assist/coordination, objective timing, ally proximity	Team metrics improve without severe individual collapse
F4	Map objectives, rotations, vision proxies, winrate	Robust against multiple opponent policies
F5	Hero-pool generalization, patch robustness	Pass full regression matrix + zero catastrophic collapse

9. Evaluation Framework — the Biggest Missing Layer

A reward curve alone cannot establish that the agent is improving. The source already calls for checkpoint-vs-checkpoint and bot-vs-agent evaluation plus rollback. [Source: Blueprint, Section 7, lines 134–140] Expand this into a formal evaluation harness.

9.1 Evaluation Tiers

Tier	Question	Metrics
Environment correctness	Is the simulator/interface reliable?	step latency, dropped messages, invalid actions, reset consistency
Learning	Is PPO learning?	episode return, policy loss, value loss, entropy, KL, explained variance
Performance	Is the agent better?	winrate, Elo, objective differential, net-worth-at-time, death rate
Robustness	Does it generalize?	opponent pool, hero pool, seeds, starting states, patch variants
Strategy quality	Did it discover useful behavior?	matched build comparisons, timing distributions, value deltas

9.2 Elo-Based League Evaluation

```
For each checkpoint C_i:
  play N games against:
    - current champion
    - historical checkpoints
    - scripted baselines
    - weak/random policy (sanity)
  update Elo/Glicko estimates
  compute confidence interval on win probability
  flag regression if lower CI crosses threshold
```

Use fixed evaluation seeds and a frozen opponent set for regression. Maintain a separate rotating evaluation set to reduce overfitting to the benchmark.

9.3 Statistical Rules

Do not gate promotion on one small batch. Record sample size, seed, opponent distribution and confidence interval. A practical first rule is to require both a point estimate threshold and a confidence-lower-bound threshold.

10. Training Loop — Exact Engineering Sequence

1. load_config(experiment.yaml)
2. seed_all(global_seed)

3. start_env_pool(N)
4. load_policy(checkpoint or init)
5. collect rollouts(T steps × N envs)
6. compute GAE + returns with terminal/bootstrap masks
7. PPO update for K epochs
8. evaluate quick suite
9. log metrics / artifacts
10. save checkpoint
11. compare with champion + historical pool
12. if regression: rollback / quarantine checkpoint
13. periodically run full evaluation matrix
14. promote curriculum only when gates pass

10.1 PPO Configuration — starting point

Parameter	Starting value	Tuning rule
LSTM hidden	256	128–512 for MVP
Rollout horizon	128–512 steps	Adjust based on temporal credit assignment
PPO epochs	3–5	Increase only if policy under-updates
Clip range	0.2	Track approximate KL
Entropy coefficient	0.005–0.02	Anneal based on entropy health
Value coefficient	0.5	Tune after reward normalization
GAE λ	0.95	Tune only after environment is stable
Discount γ	0.99	Use lower only for short-horizon tasks

These are engineering starting values, not claims of optimality. All must be experiment-configurable.

11. Repository & Code Structure

```
dota-rl/
├── configs/
│   ├── env.yaml
│   ├── ppo_f0.yaml
│   ├── ppo_f1.yaml
│   └── eval.yaml
├── env/
│   ├── protocol.py
│   ├── observation.py
│   ├── action.py
│   ├── reward.py
│   ├── masks.py
│   └── dota_adapter.py
├── transport/
│   ├── zmq_bridge.py
│   └── schemas.proto
├── agents/
│   ├── encoder.py
│   ├── policy.py
```

```
│   ├── value.py
│   ├── recurrent.py
│   └── embeddings.py
├── training/
│   ├── rollout.py
│   ├── gae.py
│   ├── ppo.py
│   ├── league.py
│   └── checkpointing.py
├── evaluation/
│   ├── harness.py
│   ├── elo.py
│   ├── baselines.py
│   ├── metrics.py
│   └── regression.py
├── analytics/
│   ├── value_events.py
│   ├── build_analysis.py
│   └── heatmaps.py
├── scripts/
│   ├── api_probe.lua
│   ├── launch_worker.py
│   └── patch_diff.py
├── dota2/scripts/vscripts/bots/
│   ├── bot_generic.lua
│   └── rl_action_handler.lua
└── tests/
    ├── unit/
    ├── integration/
    └── smoke/
```

12. Data Contracts & Versioning

Artifact	Required fields
Episode	episode_id, patch_id, schema_version, seed, start_time, outcome
Transition	obs, action, reward, next_obs, terminated, truncated, info
Checkpoint	model_hash, config_hash, git_commit, patch_id, curriculum_phase, metrics
Evaluation run	eval_version, opponent_pool_hash, seed_set, N_games, results
Patch diff	old_patch, new_patch, changed_entities, structural_changes, approval_state

Never use an unversioned observation schema in training. A patch change must produce a new data contract version whenever tensor semantics change.

13. Testing Strategy

Layer	Test
Unit	Normalization bounds; reward deltas; masks; action serialization; GAE
Protocol	Lua ↔ bridge ↔ Python round-trip; dropped/duplicate/out-of-order message handling
Environment	reset/step invariants; episode termination; deterministic seed smoke test
Model	tensor shapes; recurrent reset; masked logits; finite gradients
Training	one-batch update improves no NaNs; checkpoint save/load equivalence
Regression	Champion vs challenger on frozen benchmark
Soak	Long-running rollout stability and memory growth

14. Observability & Failure Detection

The source explicitly adds TensorBoard/W&B, checkpoint versioning, rollback and A/B testing. [Source: Blueprint, Section 7, lines 134–140] Implement the following dashboard groups:

Dashboard	Metrics
Training health	policy loss, value loss, entropy, KL, clip fraction, grad norm, explained variance
Environment health	step latency p50/p95/p99, resets/hour, invalid actions, dropped packets, desync count
Game performance	winrate, Elo, K/D, objective differential, net worth @ t, tower damage
Exploration	action entropy, unique action templates, RND error/reward
League	matchup matrix, historical wins, exploitability proxy, champion changes

15. Meta Discovery & Power-Spike Analytics

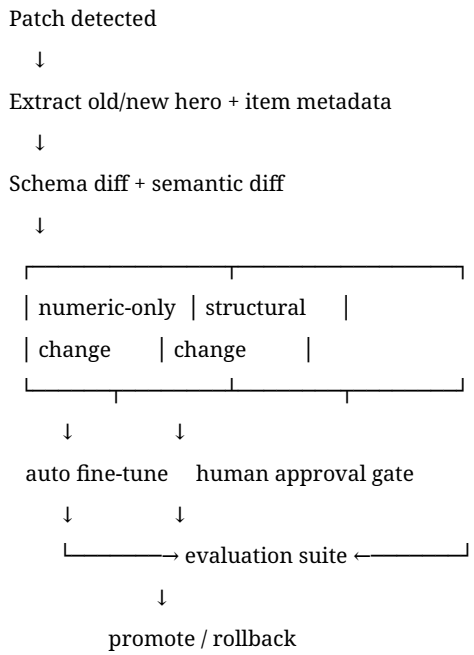
The source proposes using the value head V(s) around item purchases and combat events. [Source: Blueprint, Section 8, lines 174–175] A stronger method is event-aligned analysis: compare value before/after an event against matched states where the event did not occur.

```
For each event e at time t:
  baseline = V(s_{t-k})
  post     = V(s_{t+m})
  delta_v  = post - baseline
  collect covariates:
    hero, level, gold, enemy hero, map region, ally count,
    cooldown state, item state, objective state
  group by event type / matchup / build path
  estimate distribution, not just mean
```

This avoids interpreting a raw value jump as causal. The analytics layer should report opportunity distributions and matched comparisons, not only “best item = X”.

16. Patch Resilience Pipeline

The source proposes normalized features, embeddings, transfer learning, league refresh and automated patch-diff detection. [Source: Blueprint, Section 4, lines 96–103]



17. Compute & Capacity Planning

The source notes a very large OpenAI Five reference scale and a much smaller MVP estimate, while explicitly warning that the estimate is not a guarantee. [Source: Blueprint, Section 6, lines 123–130] Engineering should replace generic worker counts with measured throughput.

Benchmark	Measure	Decision
Environment	steps/sec/worker	How many parallel envs are feasible?
Bridge	serialization µs / MB	Is Python transport sufficient?
GPU	samples/sec, utilization	Batch size / learner count
Training	updates/hour	Expected improvement per compute hour
Eval	games/hour	How fast can checkpoints be validated?

Capacity formula: required_envs ≈ target_samples_per_second / measured_env_steps_per_second. Measure first; then budget GPUs and workers.

18. Implementation Order — from Zero to Working PoC

Milestone	Deliverable	Exit criterion
M0	API probe + environment access matrix	Known available/unavailable fields; stable reset/step
M1	F0 environment adapter	1000+ clean transitions; no deadlocks; valid actions
M2	PPO single-worker	Losses finite; policy changes; learning signal measurable
M3	PPO multi-worker	Throughput scales; no protocol corruption
M4	F1 curriculum	Basic 1v1 competence beats baseline
M5	F2 build learning	Stable power-spike / build experiments

M6	Evaluation harness	Automated checkpoint regression + Elo
M7	League self-play	Historical pool + champion/challenger promotion
M8	Meta analyzer	Event-aligned value analytics
M9	Patch pipeline	Diff → retrain → regression gate

19. MVP Definition

A credible MVP should stop at F2. The source already recommends F0–F2 as the tractable starting region and reserves full 5v5 for later curriculum phases. [Source: Blueprint, Section 5, lines 108–117]

MVP includes	MVP excludes
1v1 environment with controlled hero pool	Full 5v5 full hero pool
PPO + LSTM	Large recurrent stacks
Action masking	Complex continuous control everywhere
Reward normalization + terminal objective	Dozens of finely tuned reward terms
Fixed opponent baseline + Elo	Human-scale league
Checkpoint registry + regression tests	Fully automated patch retraining
Basic embedding support	Claim of zero-shot optimality on new heroes
Value-event analytics	Automated “meta generator” without validation

20. Acceptance Criteria

ID	Acceptance test	Pass condition
AC-01	Environment connectivity	No protocol deadlock in a 30–60 minute soak test
AC-02	Observation correctness	All required fields pass schema and range tests
AC-03	Action validity	>99% of emitted actions are valid under current mask
AC-04	PPO numerical stability	No NaN/Inf gradients or parameters in sustained training
AC-05	Learning	Performance improves across at least 3 random seeds
AC-06	Regression	No statistically significant collapse vs champion on frozen suite
AC-07	Reproducibility	Same code/config/seed produces comparable metrics within tolerance
AC-08	Meta analytics	Detected events correlate with measurable outcome changes on held-out matches
AC-09	Patch safety	Structural patch changes trigger the approval gate

21. Key Risks & Mitigations

Risk	Impact	Mitigation
Bot API lacks required state	Project blocked	API probe + simulator fallback
Low environment throughput	Training infeasible	Simplify scope; batch envs; profile; simulator
Reward hacking	False competence	Normalize + terminal objective + ablations
Invalid-action explosion	Training noise	Hierarchical heads + masks
Policy collapse	Severe regression	KL/entropy monitoring + champion rollback
Self-play cycling	No durable progress	Historical league pool + fixed eval

Patch structural mismatch	Runtime/training errors	Schema version + semantic diff gate
Overinterpreted meta signals	Bad recommendations	Matched evaluation + confidence intervals

22. Recommended Decisions vs Source Blueprint

Source decision	Recommendation
PPO + LSTM	Keep for baseline.
20k full observation reference	Do not adopt for MVP; use compact structured observation.
Embedding for new units	Keep, but test zero-shot and few-shot separately.
RND from the start	No; introduce only after baseline exploration stalls.
80/20 league split	Make configurable experiment variable.
Reward formula	Keep concept; add z-score/clip and event normalization.
C++/Rust ingestion	Defer until Python is measured as bottleneck.
Winrate >70% as phase gate	Replace with multi-metric + statistical gate.
Value-head meta extraction	Keep, but use event-aligned matched analysis.

23. First 10 Engineering Tickets

- Create repository and pinned Python/PyTorch environment with reproducible lockfile.
- Implement environment protocol types and schema_version.
- Write Lua API probe to enumerate accessible state/action fields.
- Implement Python bridge with timeout/retry/order checks.
- Implement F0 deterministic mock environment for model development independent of Dota runtime.
- Implement observation normalization and action-mask unit tests.
- Implement PPO + LSTM baseline with multi-head action policy.
- Implement rollout buffer + GAE + TBPTT.
- Implement checkpoint metadata + evaluation harness + Elo.
- Only after all above pass: connect the learner to the real Dota sandbox adapter.

24. Final Verdict

The revised blueprint is a strong research direction, but the highest-value improvement is to convert it from a "large system design" into a "measurement-driven build sequence." The architecture should therefore be:

- Environment correctness first.
- PPO/LSTM baseline second.
- Action masking and reward normalization before sophistication.
- Formal evaluation before league/RND.
- Meta analytics only after the policy is demonstrably competent.
- Patch automation only after reproducible regression testing exists.

This sequencing directly follows the blueprint's core principle—start small, prove the pipeline, then scale complexity—while making the missing engineering contracts explicit. [Source: Blueprint, Section 1, lines 28–32]

Appendix A — Configuration Skeleton

```
experiment:
  name: dota_rl_f1_baseline
  seed: 42
  patch_id: sandbox_current
  curriculum_phase: F1

environment:
  num_envs: 32
  step_timeout_ms: 200
  obs_schema_version: 1
  action_schema_version: 1

model:
  encoder_dim: 256
  lstm_hidden: 256
  lstm_layers: 1
  entity_attention_heads: 4

ppo:
  gamma: 0.99
  gae_lambda: 0.95
  clip_range: 0.2
  epochs: 4
  minibatch_size: 2048
  entropy_coef: 0.01
  value_coef: 0.5
  max_grad_norm: 0.5

reward:
  normalize: true
  clip: 5.0
  team_blend_tau: 0.0
  terminal_win: 10.0

exploration:
  entropy_anneal: true
  rnd_enabled: false

league:
  enabled: false
  historical_ratio: 0.2

evaluation:
```

games_per_opponent: 50

deterministic: true

regression_threshold: 0.05

Appendix B — Source Traceability

Source section	Used in this document
Scope & legal	Sections 1–2
Tech stack / pipeline	Sections 3–4
Observation/action / embeddings	Sections 4–5
Exploration / reward	Sections 6–7
Patch resilience	Section 16
Curriculum	Sections 8, 19
Compute	Section 17
Evaluation / observability	Sections 9, 14, 20
Roadmap	Section 18